proofmint

FUTURESTACK INKLOOM HACKATHON / ROUND 3

Nothing ships unproven.

R3 FINAL IMPLEMENTATION REPORT

A human-governed verification layer between content creation and publication. Findings show their evidence status; repairs remain previews; clearance remains a reviewer decision.

EVIDENCE-LOCKED FINDINGS

INTENT-PRESERVING REPAIR

AUTOMATED EVALUATION + AMBIGUITY RADAR

ORIGINAL PROBLEM STATEMENT / ROUND 2

"The problem we are addressing is the lack of a reliable, traceable verification step between content creation and publication."

TEAM EMIC TECH

LOCAL R3 CLOSEOUT | 26 OCTOBER 2026

Project overview

Current Round 3 status is based on the local source tree and executed checks. The Round 2 report is used for original problem framing, branding, and documented team details - not as evidence of current features or tests.

Original problem statement. Proofmint addresses the lack of a reliable, traceable verification step between content creation and publication. Brand guidance and approved claims can be spread across documents and shared sources. Creators may use outdated information or interpret rules differently; fluent copy can still contain unsupported claims. Reviewers need the exact text span, source, evidence status, and a human-controlled next step.

Product. Proofmint is a pre-publication brand verification workspace for marketing content. It organizes guideline candidates, approved facts and terminology, runs deterministic literal checks, records review decisions, previews supported corrections, and gates clearance on unresolved actionable findings. The product identity remains AI-powered brand verification; the current implementation does not call an LLM or perform semantic or legal analysis.

Intended users

User group	Need served
Content creators and agency partners	Specific feedback that can be reviewed and corrected without unnecessary rewriting.
Brand and compliance reviewers	Traceable findings, evidence status, and authority to resolve, dismiss, flag, or hold.
Marketing and brand leaders	Visibility into recurring rule-linked review friction and opportunities to clarify guidance.

The scale of these needs and product impact have not been measured in real-world use.

Seven-step product workflow

The original R1 sequence is retained in order. Status describes current local functionality, not the older Round 2 status.

Stage	Current implementation
1. Configure approved brand knowledge	Upload bounded guideline PDFs and import facts and terminology CSVs. PDF bytes and full extracted text are not retained.
2. Review and approve the rules	Candidate rules, facts, and terms start pending review. People approve or reject records; extraction never approves rules automatically.
3. Submit marketing content	A creator submits copy in the active workspace. Full copy remains client-side for the active flow; API persistence is limited to a hash and finding metadata.
4. Run the content audit	Deterministic checks find explicitly quoted restricted phrases, preferred-terminology mismatches, and exact approved-fact text matches.
5. Verify the findings	A separate verifier checks spans, evidence identifiers, approval, workspace, source metadata, and explanation. Outcomes are verified, uncertain, or rejected.
6. Correct and review	Only verified preferred-terminology suggestions are used for repair preview. Humans can edit or decide; copy is not silently rewritten or published.
7. Record clearance and improve the guidelines	Clearance is explicit. Pending or flagged actionable findings block clearance. Review events can feed rule-level aggregates.

R3 improvements

1. Evidence-Locked Findings

A separate backend verifier runs after finding generation and before persistence. It validates the exact submitted-text span and phrase, requires exactly one evidence reference, loads the current record, checks approval and workspace ownership, and validates available provenance and explanation. Reason codes and human-readable verification explanations are persisted and returned by typed API responses.

Outcome semantics: verified means supported literal evidence and span checks passed, not that an arbitrary claim is true. Uncertain means available source material cannot independently establish the rule finding, especially because original PDFs and full page text are not retained. Rejected identifies invalid spans, references, approval state, workspace, or metadata.

Uncertain or rejected findings are not presented as verified.

2. Intent-Preserving Repair

The repair preview endpoint requires the exact audited copy hash and rechecks current evidence and spans. It replaces only verified preferred-terminology spans, preserves surrounding content, and reports before/after text, source, reason, and replacement details. Changed input, stale spans, or invalid evidence are refused; if no safe repair exists, no replacement is made.

Acceptance is a separate human decision requiring original submitted content and a fresh evidence check. The API does not mutate the submission, publish a revision, or clear an audit. A user must intentionally edit/use a preview and run a new audit. No repair is offered for restricted phrases or unsupported claims.

3. Automated Evaluation and Ambiguity Radar

The reproducible evaluation command runs eight version-controlled synthetic cases in an in-memory SQLite database. It measures normalized finding precision/recall, evidence outcome agreement, two-run signature consistency, and exact repair validity. The measured precision 1.0, recall 1.0, and consistency 1.0 rates describe only this small synthetic set; they do not estimate customer, production, or real-world accuracy.

The workspace API aggregates persisted decisions by currently approved rule. It reports dismissals, flags, accepted suggestions, recorded reasons, and score = (dismissals + flags) / all rule-linked decision events. A repeated signal requires at least three events and score >= 0.5. Lower-volume rules remain insufficient data. Reviewer identities are unavailable, so the system does not claim interpersonal disagreement.

Architecture and data boundaries

Layer	Implementation
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS; Studio has the seven-stage workflow, evidence labels, repair preview, review controls, and API-backed radar.
API	Python 3.13, FastAPI, Pydantic v2; typed routes for audit, decisions, repair preview, clearance, and radar.
Persistence	SQLAlchemy 2, PostgreSQL, Psycopg 3, Alembic. Revisions 0001-0005. PostgreSQL engine is lazy and missing configuration is an explicit error.
Document processing	PyMuPDF extracts bounded PDF text in memory. Metadata, candidates, excerpt, page, and offsets persist; PDF bytes and complete page text do not.
Testing	Pytest with explicitly isolated SQLite fixtures; opt-in PostgreSQL integration tests; deterministic labeled JSON evaluation cases.

The API stores content hashes, matched phrases, excerpts, decision reasons, and audit history, not complete submitted copy. Free-text decision reasons should not include confidential content.

Human authority and governance

- Approval: candidates and imported records start pending; people approve or reject them.
- Verification: status reports only checks established by literal evidence; uncertain source support remains explicit for review.
- Repair: suggestions are previews. A person chooses whether to edit or use a revision; no background rewrite occurs.
- Clearance: a human records clear/hold; pending or flagged actionable findings block clearance.
- Guidelines: radar informs review but never changes rules; changes remain human-controlled.

Testing and verified results

Results are from actual local runs on 26 October 2026. The PostgreSQL integration run is separate from the default backend run.

Check	Observed result
Python imports	Passed; app.main and audit modules resolved from services/api/app.
Focused R3/API tests	24 passed.

Check	Observed result
Persisted audit workflow test	1 passed.
Full backend suite	80 passed, 4 skipped in 8.27 seconds; 84 items collected. Four PostgreSQL tests were opt-in and skipped in this invocation.
PostgreSQL integration tests	4 passed in 4.03 seconds after migrations 0001-0005 in disposable loopback schema pm_r3_test_80cfa58fed73; test wrapper reported dropping it.
Synthetic evaluation	8 cases: TP 4, FP 0, FN 0; precision 1.0, recall 1.0; evidence 2 verified, 1 uncertain, 1 rejected; expected evidence outcomes 4/4; repair validity 5/5 including 2 safe refusals; consistency 8/8.
Frontend workspace tests	3 passed; 0 failed.
Frontend lint and TypeScript	npm run lint passed without warnings/errors; npx tsc --noEmit exited 0.
Production build	npm run build passed outside sandbox: compiled, type-checked, generated all 9 static pages, completed build tracing, exit code 0.

An earlier sandboxed Next build failed with Windows EPERM while workers created .next directories. No Node process held .next and direct Node mkdir/rmdir worked. The build succeeded outside the sandbox without a source diagnostic, indicating a sandbox child-worker filesystem restriction, not a stale or locked output directory.

Browser E2E was not run to completion. The previous in-app browser timed out at localhost; Windows browser handoff then stopped because it could not confidently determine the current URL. API-level ASGI tests are not browser E2E.

The eight-case synthetic evaluation does not demonstrate real-world or production accuracy.

Database safety and verification scope

The integration tests used a uniquely named loopback schema. Their wrapper reported dropping pm_r3_test_80cfa58fed73. A separate schema, pm_r3_ui_6f22bc02fb77, was created for a planned UI run. After its API process received a stop signal, a read-only existence query found it still present. It has been left untouched because cleanup could not be confirmed. No other schemas were enumerated or changed. The default/public schema and production Neon were not migrated.

Known limitations

- No authentication or tenant authorization; UUID workspace scoping is not a security boundary.
- No semantic claim verification, legal analysis, or LLM-backed audit. Checks are deterministic and literal.
- Original guideline PDFs and complete extracted text are not retained, so guideline findings remain uncertain when the source cannot be independently re-read.
- Repair is limited to preferred terminology and does not score persuasive intent, cadence, or broader minimal-token quality.
- Evaluation has eight synthetic cases; its scores do not support real-world accuracy claims.
- Browser E2E did not complete; no browser workflow is reported as passed.
- No production deployment, production migration, or production environment change was performed.

Roadmap

- Implement authentication, workspace authorization, reviewer identity, and access controls before production use.
- Broaden labeled evaluation with representative rules, terms, source spans, and human-rated repair quality.
- Design privacy-reviewed source retention if original-PDF verification is required.
- Improve revision provenance without retaining full copy; review decision-reason retention/redaction.
- Add observability, rate limits, backups, and further isolated PostgreSQL coverage.
- Complete browser E2E and assistive-technology testing across desktop and mobile.
- Consider model-assisted checks only behind evidence provenance and evaluation gates; use reviewed legal sources for jurisdiction-specific guidance.

Team and documented responsibilities

The Round 2 report documents team EMIC TECH and institution Vivekananda Global University (NIAT), B.Tech. / Software Engineering. Roles below are historical Round 2 workstream ownership and do not establish who performed particular R3 implementation tasks.

Participant	Round 2 documented role
Keshav Modi	AI Engineering Lead: guideline compiler, retrieval-grounded analysis, claim verification, Evidence Verifier.
Akshara Jain	Backend & Platform Lead: APIs, database, versioned rules/facts, workflow, access control, clearance records.
Yuvraj Rathore	Frontend & UI/UX Lead: submission interface, finding annotations, corrections, reviewer controls.
Dhananjay Dagar	Product & Research Lead: research-source verification, requirements, scope, hackathon criteria.
Sparsh Shrimal	Data & Evaluation Support: synthetic brand, labeled examples, seeded violations, evaluation records.
Tanmay Singh Panwar	QA, Documentation & Demo Support: tests, documentation, setup, known limitations, demonstration.

No individual R3 contribution is asserted without separate confirmation. The Round 2 report lists integration, testing, and final review as shared responsibilities.

Project links and references

Source repository: https://github.com/ViSiONiX07/Proofmint

Frontend: https://proofmint-rust.vercel.app/

Backend: https://proofmint-q9ss.onrender.com/

Swagger API: https://proofmint-q9ss.onrender.com/docs

Health endpoint: https://proofmint-q9ss.onrender.com/api/v1/health

Links are preserved from project materials. Their inclusion is not evidence that live services were tested or changed. Supporting R3 Markdown documents are in docs/round-3/. The supplied Round 2 PDF is historical reference for problem framing, brand, and team information; its old feature/test status was not carried forward.